CSCI 532 Project Paper Part 2: Conventional Federated Learning Under Data Poisoning

Tyriz Newton
Rochester Institute of Technology
tn1207@rit.edu

Kai Fan
Rochester Institute of Technology
kf5601@rit.edu

Abstract

Federated learning trains a shared model without moving the data off of each client, but it is still open to attacks like data poisoning. In this paper we built a conventional federated learning setup with FedAvg and tested it on handwritten digits with 20 clients, with and without a label flipping attack. Without an attack the average accuracy reached 76.0%, but with 10 of the 20 clients poisoned it only reached 26.7%. This shows that conventional federated learning has no protection against poisoned clients, which leads into the PID defense in Part 3.

1. Introduction

Federated learning (FL) lets many clients train one shared model without sending their data to a central server. Each client trains the model on its own data and sends only the model weights back, and the server combines them using an aggregation method. The most common method is Federated Averaging, or FedAvg [1], which takes a weighted average of the client models. Even though the data stays on the clients, FL is still open to attacks. A compromised client can send bad updates, and conventional FL has nothing in place to catch them.

In this part of the project we built a conventional FL setup and tested how it holds up against data poisoning. Specifically, we used a label flipping attack [6], where a compromised client trains on data with the wrong labels. We first ran the setup normally to get a baseline, and then ran it again with some of the clients poisoned. Since we are Group 5, 50% of the clients (10 of 20) were poisoned in the main attack run. These results will be used in Part 3 to see how much the PID defense improves on conventional FL.

2. Experimental Setup

2.1 Dataset

For the data, we used the instructor provided subset of FEMNIST [4], which is made up of handwritten digits split evenly among 20 clients. Each client has its own folder with one subfolder for each digit from 0 to 9. The images are loaded as grayscale and the pixel values are scaled to be between 0 and 1. Since this is the instructor provided dataset, we did not need a separate data preparation strategy.

Each client splits its own data into 80% for training and 20% for evaluation. We used a fixed seed (42) so the split is the same every time. This means every client is evaluated only on its own data, and the evaluation sets are small, about 100 images each. This is worth keeping in mind when reading the per client graphs in Section 4.

2.2 Model

The model we used is a convolutional neural network (CNN) built in PyTorch [3]. As stated in Part 1, we first tried a multilayer perceptron, but the accuracy was poor, so we switched to the CNN. The CNN has two convolutional layers with 32 and 64 filters, and each one is followed by a ReLU and a max pooling layer, which shrinks the 28 by 28 image down to 7 by 7. After that there is a dense layer with 128 units, a dropout layer of 0.2, and a final layer with 10 outputs, one for each digit. Every client uses the same model, and the server sends the same starting weights to all of them.

2.3 Federated Learning Configuration

We used the Flower framework [2] to simulate 20 clients on one machine, and every client takes part in every round. The server uses FedAvg, which weights each client's update by how many training samples it has. In each round, a client trains for one epoch with the Adam optimizer [5] and then evaluates the new global model on its own evaluation data. We ran 10 rounds for the main experiments. The settings are listed in Table 1.

Table 1. Federated learning configuration.

Parameter	Value
Clients	20 (all participate every round)
Aggregation rounds	10 (main experiments)
Aggregation strategy	FedAvg (sample weighted)
Local training	1 epoch per round, batch size 32
Optimizer	Adam, learning rate 0.001
Loss function	Cross entropy
Train / evaluation split	80% / 20%, seed 42

2.4 Attack Model

The attack we used is label flipping, which is a type of data poisoning. On a poisoned client, every label y is changed to (y + 1) mod 10, so a 0 is labeled as 1 and a 9 is labeled as 0.

This change is made when the client's data is loaded, before it is split, so both the training and evaluation data of that client are poisoned. This follows the project requirement that a poisoned client is evaluated on its own poisoned data.

For the main Group 5 experiment, clients 0 to 9 were poisoned, which is 50% of the clients, with a flipping rate of 100%. For the severity sweep, the flipping rate is the fraction of a poisoned client's samples that get the changed label.

2.5 Metrics Collected

After every round, each client reports its loss and accuracy for the new global model, and the server saves them. From these we get the four required metrics, which are loss per client, average loss, evaluation accuracy per client, and average evaluation accuracy. The averages are taken over all 20 clients. Everything is saved to CSV files and plotted.

2.6 Experiments

We ran three experiments, which are listed in Table 2. The first is the baseline with no attack, the second is the Group 5 attack, and the third is a sweep that changes how many clients are poisoned and how many of their labels are flipped.

Table 2. Experiments performed.

No.	Experiment	Poisoned clients	Flip rate, rounds
1	No attack (baseline)	0 of 20	None, 10
2	Group 5 attack	10 of 20 (50%)	100%, 10
3	Severity sweep	4, 10, 16 of 20	20, 50, 80, 100%, 5

3. Code and User's Guide

The code is split into three files, which are shown in Table 3. The results are saved into the results/no_attack and results/attack folders.

Table 3. Code structure.

File	Purpose
src/dataset.py	Loads one client's images, applies label flipping when poison is enabled, and builds the 80/20 train and evaluation loaders.
src/model.py	DigitCNN model definition.
src/train.py	Flower client, FedAvg strategy that logs per client metrics, experiment runner, CSV export, plotting, and main entry point.

To run the code, Python 3 is needed along with torch, torchvision, flwr with the simulation extra (which installs Ray), pandas, numpy, matplotlib, and pillow. The dataset should be placed in a folder called femnist_dataset next to the src folder, so that each client is at femnist_dataset/client_<id>/<digit>/*.png. Running python src/train.py from the project root will then run all of the experiments in order. It uses a GPU if one is available and runs on the CPU otherwise. The poisoned client IDs are set in main(), and they can be changed to test other attack setups.

Each run saves loss_per_client.csv, acc_per_client.csv, avg_loss.csv, and avg_acc.csv, along with the four matching history_*.png plots. The severity sweep also saves attack_severity.csv and attack_severity_dependence.png.

While running the simulation with Flower and Ray on Windows, we ran into two problems and used solutions from Stack Overflow to fix them. First, Ray runs the clients in separate threads, and these threads tried to use the default Matplotlib GUI backend, which caused crashes and graphs that showed all zeros. Following a Stack Overflow answer, we set Matplotlib to the non GUI Agg backend before importing pyplot in train.py. Second, the string client IDs that Ray uses for its client proxies caused hash mismatches when the results were collected. Following another Stack Overflow answer, each client now returns its integer ID in its metrics dictionary, and the server uses that ID to record the results.

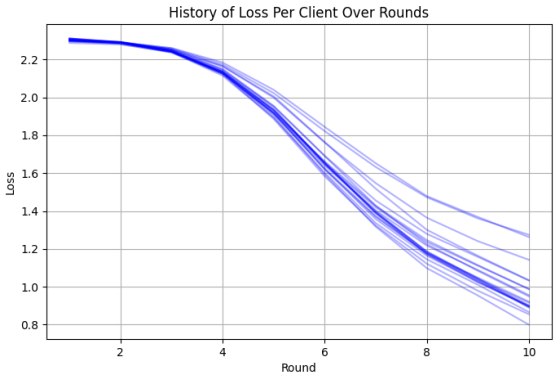
Two other changes were made because of Ray. The src folder is added to the Python path so the worker processes can find dataset.py and model.py, and image paths are stored as strings instead of Path objects so they can be serialized. All of these fixes are marked with comments in the code.

4. Results

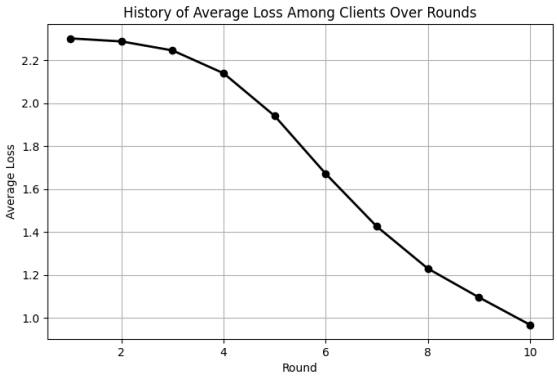
4.1 Evaluation Without an Attack

Without an attack, the model trained as expected, as shown in Figure 1. The average loss went from 2.303 in round 1 down to 0.968 in round 10, and the average accuracy went from 12.2% up to 76.0%. The starting loss of about 2.30 is the same as ln(10), which is what a model that is just guessing between 10 digits would get. The accuracy improved quickly between rounds 3 and 7, but then slowed down to only 2 to 4 points per round near the end. This means the model was starting to level off, but it had not fully converged after 10 rounds.

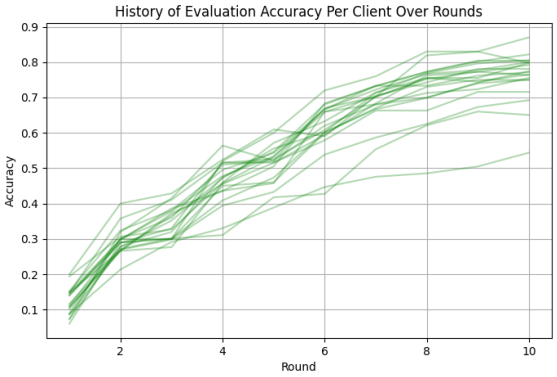
The per client graphs in Figures 1a and 1c stay close together. The final accuracies of the clients ranged from 54.4% to 87.0%, and the final losses ranged from 0.80 to 1.27. Since each client only has about 100 evaluation images, a difference of a few points between clients can come from chance alone. Overall, all of the clients acted in a similar way, so this run works well as a baseline.



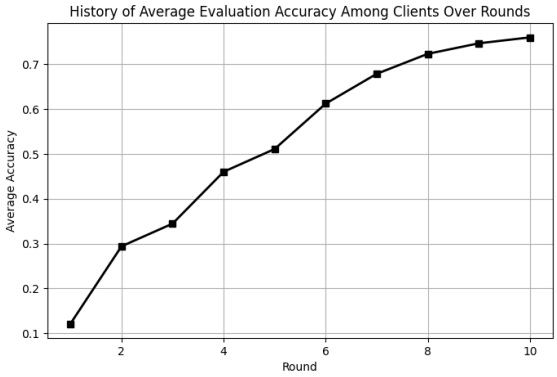
(a) Loss per client



(b) Average loss



(c) Evaluation accuracy per client



(d) Average evaluation accuracy

Figure 1. Conventional FL without an attack.

4.2 Evaluation With an Attack (Group 5)

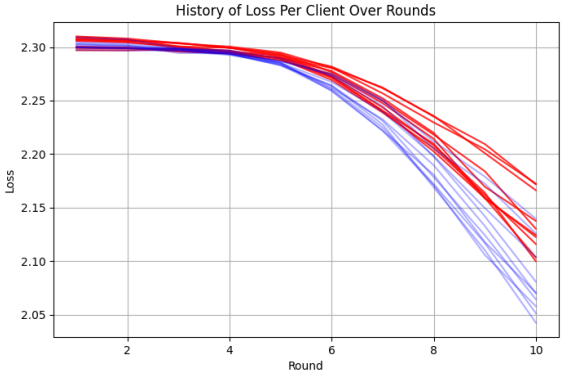
When 50% of the clients were poisoned, the training was badly damaged, as seen in Figure 2. The average loss barely changed for the first five rounds (2.303 to 2.288) and ended at 2.108, which is still close to the 2.30 of random guessing. The average accuracy only reached 26.7%, compared to 76.0% in the clean run. The accuracy was also unstable, since it dropped in rounds 3 and 5 before going up faster after round 6. Table 4 compares the poisoned and benign clients in the last round.

Table 4. Final round metrics by client group.

Metric	Poisoned	Benign	All
Avg accuracy	25.8%	27.7%	26.7%
Avg loss	2.134	2.081	2.108

The poisoned clients ended with a slightly higher loss and lower accuracy than the benign clients, and the red loss lines in Figure 2a are above the blue ones starting around round 6. However, the poisoned and benign accuracy lines in Figure 2c overlap a lot. This is what we expected for this setup. FedAvg averages the updates of 10 honest clients and 10 poisoned clients, so the global model ends up as a mix that does not match the true labels or the flipped labels well. This means it performs poorly for every client, and the benign clients are hurt almost as much as the poisoned ones.

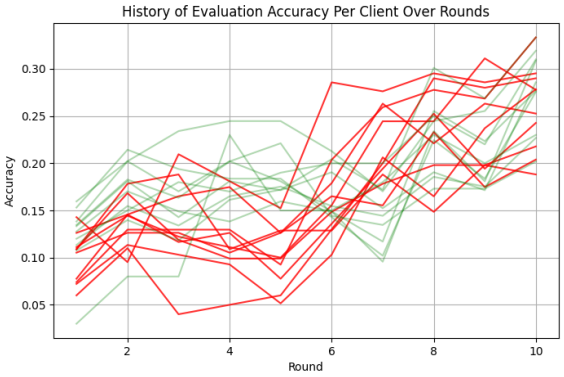
We checked that the poisoning works because the flipped labels are put into the client data before it is split, each client is evaluated on its own data, and the accuracy dropped a lot compared to the clean run. Loss is the better way to tell the two groups apart, since the accuracy difference of about 2 points is smaller than the noise we would expect from only about 100 evaluation images per client.



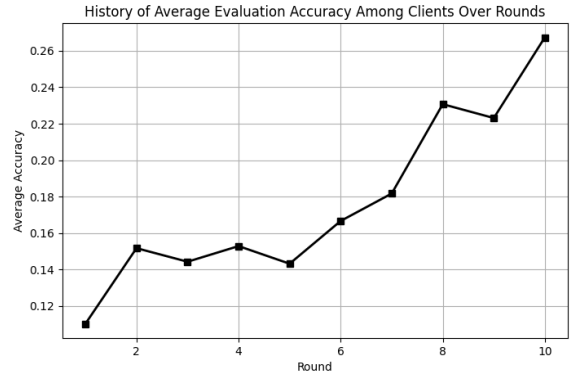
(a) Loss per client (red: poisoned, blue: benign)



(b) Average loss



(c) Accuracy per client (red: poisoned, green: benign)



(d) Average evaluation accuracy

Figure 2. Conventional FL with 10 of 20 clients poisoned (100% label flipping).

4.3 Comparison of the Two Runs

Figure 3 and Table 5 compare the two runs. They both start at about the same place (11% to 12% accuracy and a loss of 2.30), but they split apart right away. By round 2, the clean run was at 29.4% accuracy and the attacked run was at 15.2%. By round 10, the difference was 49.3 percentage points in accuracy and 1.14 in loss. The attacked run was still improving at round 10, so it might get better with more rounds, but it would still be far behind the clean run. Conventional FL has no way to remove the harmful clients, so they keep affecting the global model.

Table 5. Average metrics per round.

Round	Acc, clean	Acc, attack	Loss, clean	Loss, attack
1	12.2%	11.0%	2.303	2.303
2	29.4%	15.2%	2.288	2.302
3	34.5%	14.4%	2.247	2.299
4	46.0%	15.3%	2.140	2.296
5	51.1%	14.3%	1.942	2.288
6	61.2%	16.7%	1.672	2.271
7	67.9%	18.2%	1.426	2.242
8	72.3%	23.1%	1.230	2.203
9	74.7%	22.3%	1.096	2.156
10	76.0%	26.7%	0.968	2.108

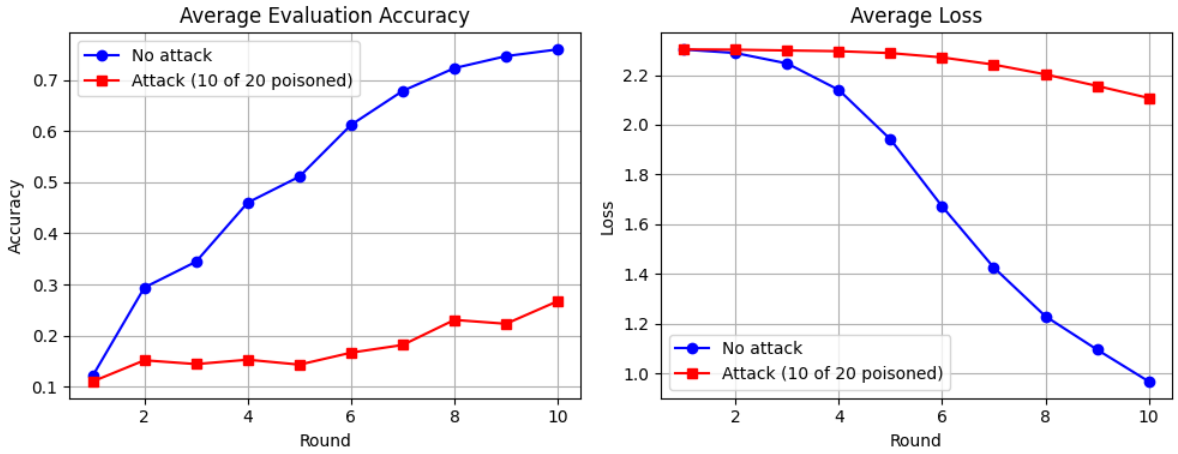


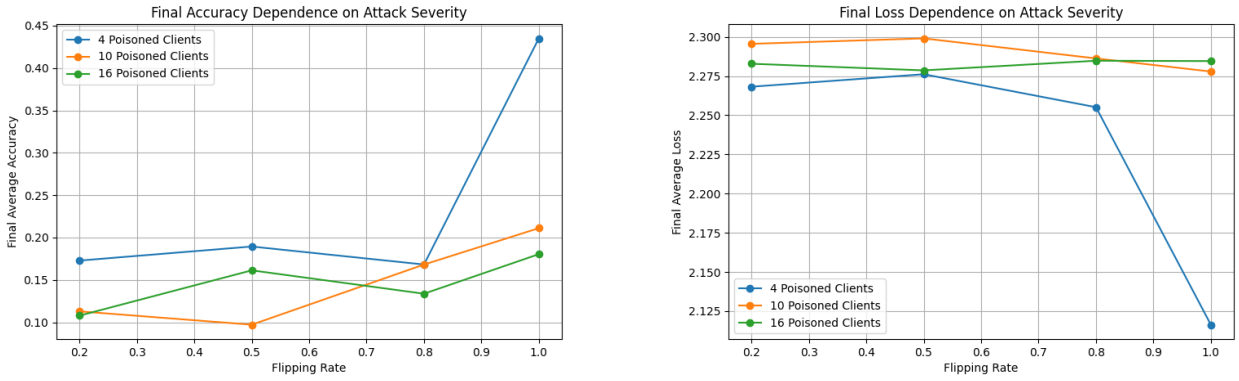
Figure 3. Average accuracy and average loss with and without the attack.

4.4 Dependence on Attack Severity

Figure 4 and Table 6 show the results of the severity sweep, which used 4, 10, and 16 poisoned clients with flipping rates of 20%, 50%, 80%, and 100%. The sweep ran for 5 rounds, which is fewer than the main experiments, so the numbers cannot be compared directly to Table 5.

Table 6. Attack severity sweep results.

Poisoned	Flip rate	Final loss	Final acc
4	20%	2.268	17.3%
4	50%	2.276	18.9%
4	80%	2.255	16.8%
4	100%	2.116	43.5%
10	20%	2.296	11.3%
10	50%	2.299	9.7%
10	80%	2.286	16.8%
10	100%	2.278	21.1%
16	20%	2.283	10.8%
16	50%	2.279	16.1%
16	80%	2.285	13.4%
16	100%	2.285	18.0%



(a) Final accuracy

(b) Final loss

Figure 4. Final average accuracy and loss versus flipping rate for 4, 10, and 16 poisoned clients.

With 10 or 16 poisoned clients, the final accuracy stayed between 9.7% and 21.1% and the final loss stayed between 2.278 and 2.299. This is close to random guessing (10% accuracy and a loss of 2.303), so the model learned very little in these runs. The only run that learned noticeably was 4 poisoned clients with a flipping rate of 100%, which reached 43.5% accuracy and a loss of 2.116. This suggests that 16 honest clients can keep the training on track when only a small group is poisoned.

The results did not change with the flipping rate in the way we expected. With 4 poisoned clients, the accuracy was 16.8% at an 80% flipping rate but 43.5% at 100%, and with 10 poisoned clients it went up from 11.3% to 21.1% as the flipping rate went up. We do not think these small differences

are reliable, because each result is a single run, the values are close to random guessing, and each client is evaluated on only about 100 images. One idea is that a 100% flip is a single consistent relabeling that all of the poisoned clients agree on, while a partial flip makes the labels of one client contradict each other. We did not test this, and it would take repeated runs with different seeds. What the sweep does show is that once 50% or more of the clients are poisoned, conventional FL does not learn within the available rounds, no matter the flipping rate.

5. Discussion and Limitations

These experiments show that FedAvg offers no protection against label flipping. With half of the clients poisoned, the final accuracy dropped from 76.0% to 26.7%, and the benign clients suffered almost as much as the poisoned ones because they all receive the same global model. This is why the defense in Part 3 is needed, since it will try to find the poisoned clients and leave them out of the aggregation.

There are some limits to these results. Each setup was only run once, and the training data is shuffled without a fixed seed, so we did not measure how much the results change from run to run. The evaluation sets are also small (about 100 images), so the per client accuracy is noisy. The main experiments were limited to 10 rounds, and the attacked run was still improving at the end, while the sweep used fewer rounds. Finally, we only tested one kind of attack, which is a label shift on the first 10 clients. Since the data is IID, which clients are poisoned should not matter much, but other attacks like random label flipping or model poisoning were not tested.

6. Work Distribution

Table 7 shows who worked on each part and how many hours were spent.

Table 7. Work distribution.

Member	Responsibilities	Hours
Tyriz Newton	Attack experiments: label flipping poisoning, the Group 5 attack run, the severity sweep, attack plots and CSV files, and interpretation of the attack results.	10
Kai Fan	No attack experiments: the clean baseline run, no attack plots and CSV files, and interpretation of the baseline results.	10
Both	Shared code base in src and report writing.	18

7. References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS), 2017.

[2] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[3] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[4] S. Caldas et al., "LEAF: A benchmark for federated settings," arXiv:1812.01097, 2018.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[6] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in Proc. European Symp. Research in Computer Security (ESORICS), 2020, pp. 480–501.